Solution: Rearrange String

Problem Statement

Given a string, find if its letters can be rearranged in such a way that no two same characters come next to each other.

Example 1:

```
Input: "aappp"
Output: "papap"
Explanation: In "papap", none of the repeating characters come next to
each other.
```

Example 2:

```
Input: "Programming"
Output: "rgmrgmPiano" or "gmringmrPoa" or "gmrPagimnor", etc.
Explanation: None of the repeating characters come next to each other.
```

Example 3:

```
Input: "aapa"
Output: ""
Explanation: In all arrangements of "aapa", atleast two 'a' will come
together e.g., "apaa", "paaa".
```

Solution

This problem follows the `Top ‘K’ Numbers` pattern. We can follow a greedy approach to find an arrangement of the given string where no two same characters come next to each other.

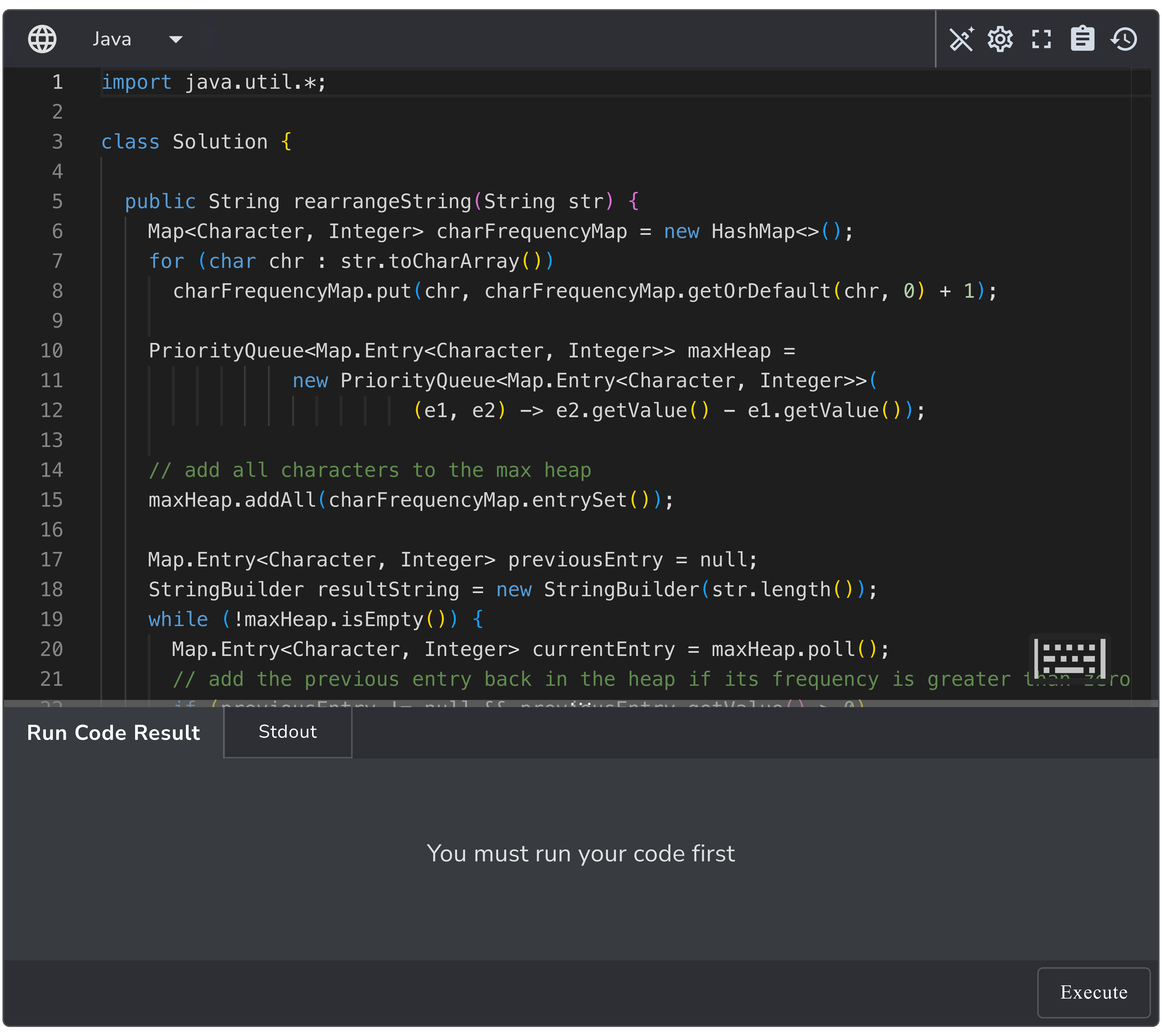
We can work in a stepwise fashion to create a string with all characters from the input string. Following a greedy approach, we should first append the most frequent characters to the output strings, for which we can use a **Max Heap**. By appending the most frequent character first, we have the best chance to find a string where no two same characters come next to each other.

So in each step, we should append one occurrence of the highest frequency character to the output string. We will not put this character back in the heap to ensure that no two same characters are adjacent to each other. In the next step, we should process the next most frequent character from the heap in the same way and then, at the end of this step, insert the character from the previous step back to the heap after decrementing its frequency.

Following this algorithm, if we can append all the characters from the input string to the output string, we would have successfully found an arrangement of the given string where no two same characters appeared adjacent to each other.

Code

Here is what our algorithm will look like:



```java
import java.util.*;

class Solution {

  public String rearrangeString(String str) {
    Map<Character, Integer> charFrequencyMap = new HashMap<>();
    for (char chr : str.toCharArray())
      charFrequencyMap.put(chr, charFrequencyMap.getOrDefault(chr, 0) + 1);

    PriorityQueue<Map.Entry<Character, Integer>> maxHeap =
            new PriorityQueue<Map.Entry<Character, Integer>>(
                (e1, e2) -> e2.getValue() - e1.getValue());

    // add all characters to the max heap
    maxHeap.addAll(charFrequencyMap.entrySet());

    Map.Entry<Character, Integer> previousEntry = null;
    StringBuilder resultString = new StringBuilder(str.length());
    while (!maxHeap.isEmpty()) {
      Map.Entry<Character, Integer> currentEntry = maxHeap.poll();
      // add the previous entry back in the heap if its frequency is greater than zero
```

You must run your code first

Time Complexity

The time complexity of the above algorithm is O(N*logN)O(N∗logN) where ‘N’ is the number of characters in the input string.

Space Complexity

The space complexity will be O(N), as in the worst case, we need to store all the ‘N’ characters in the **HashMap**.